Classification & Logistic Regression

A Practical Handbook for Kaggle Classification Projects

Python · NumPy · Pandas · Matplotlib · Scikit-learn

From raw data to a working, evaluated model — the concepts, math, and code you need to work independently on real datasets.

Compact Practical Guide

Contents

Introduction

This handbook is a compact, practical guide to **classification** and **logistic regression** using the standard Python data-science stack. It is not a textbook — it is a working reference designed so that after one read-through, you can open a new Kaggle classification dataset and independently take it from raw CSV to an evaluated, tuned model.

Each important idea in this handbook follows the same rhythm:

What it is → Why it matters → Formula/intuition → Small example → Python code → How to read the result.

Keep this mental model in mind as you move through the sections — it is the thread that connects everything:

```
DATA → INSPECT → CLEAN → UNDERSTAND FEATURES → CLASSIFICATION TYPE
→ BINARY / MULTICLASS → LOGISTIC REGRESSION → LINEAR SCORE
→ SIGMOID / SOFTMAX → PROBABILITY → THRESHOLD / CLASS
→ CONFUSION MATRIX → EVALUATION → OVERFITTING CHECK
→ REGULARIZATION → MODEL COMPARISON → KAGGLE PROJECT
```

1. Dataset Understanding: NumPy & Pandas Basics

Every classification project starts the same way: load the data, and understand it before touching any model. Two libraries make this possible.

- **NumPy** gives us fast, vectorized numerical computation — arrays instead of Python loops. Almost every ML library (including scikit-learn) is built on top of it.
- **Pandas** gives us **DataFrames** — labeled, spreadsheet-like tables that make loading, cleaning, and exploring tabular data straightforward.

```python
import numpy as np
import pandas as pd

df = pd.read_csv("dataset.csv")   # loads a CSV file into a DataFrame
```

`pd.read_csv()` reads a comma-separated file from disk (or a URL) and returns a `DataFrame` — the object you will work with for the rest of the pipeline.

1.1 First Look at the Data

Before any modeling, always run these three commands. They answer three different questions:

Command	What it shows	Why we use it	Decision it helps with
`df.head()`	First 5 rows of the data	Sanity-check that the file loaded correctly and see actual values	Spot obviously wrong columns/formats early
`df.info()`	Column names, data types, non-null counts, memory usage	Reveals data types and **missing values** at a glance	Decide which columns need type conversion or cleaning
`df.describe()`	Count, mean, std, min, quartiles, max for numeric columns	Summarizes the distribution of each numeric feature	Spot outliers, skew, or impossible values (e.g. negative age)

```python
df.head()
df.info()
df.describe()
```

1.2 Handling Missing Values and Unneeded Columns

Real datasets are rarely clean. These four methods cover most situations:

Method	Purpose
`df.isnull()`	Returns `True`/`False` per cell — usually chained as `df.isnull().sum()` to count missing values per column
`df.drop(columns=[...])`	Removes columns (e.g. IDs, names) or rows that add no predictive value
`df.dropna()`	Removes rows (or columns) that contain missing values entirely
`df.fillna(value)`	Replaces missing values with something sensible (mean, median, mode, or a constant)

```
df.isnull().sum()                          # count missing values per column
df = df.drop(columns=["customer_id"])      # drop an identifier column — no predictive value
df["age"] = df["age"].fillna(df["age"].median())   # fill missing numeric values
df = df.dropna(subset=["target"])          # drop rows where the target itself is missing
```

A simple rule of thumb: **fill** when a column is useful but has a few gaps; **drop** the column when it is mostly missing or irrelevant (like an ID); **drop the row** only when very few rows are affected.

1.3 Features, Target, and Feature Types

Every supervised learning problem splits the table into two parts:

- **X (features)** — the input columns the model learns from (e.g. age, income, tenure).
- **y (target)** — the column we want to predict (e.g. churn: yes/no).

```
X = df.drop(columns=["target"])
y = df["target"]
```

Features and targets both come in different flavors, and this affects how you preprocess and which problem type you're solving:

Type	Examples	Notes
Numerical feature	age, income, tenure	Can be used directly; often benefits from scaling
Categorical feature	gender, city, plan type	Must be encoded (e.g. one-hot) before modeling
Binary target	churn: 0/1	→ **Binary classification**
Multiclass target	species: setosa/versicolor/virginica	→ **Multiclass classification**

2. Statistical Foundation for Classification

Before modeling, a handful of statistics help you understand your data and how features relate to each other and to the target. They build on one another in a single chain:

`Mean → Deviation from mean → Variance → Covariance → Correlation → Understanding relationships between features`

We'll use one small running example: exam **study hours** vs **score**, for 5 students.

Student	Hours (X)	Score (Y)
1	1	50
2	2	55
3	3	65
4	4	70

Student	Hours (X)	Score (Y)
5	5	80

Statistic	Intuition	Formula	Meaning of the result
Mean ($\bar{x}$)	The "typical" value	$\bar{x} = \frac{1}{n}\sum x_i$	Center of the data
Variance	How spread out values are	$Var(X) = \frac{1}{n}\sum(x_i - \bar{x})^2$	Larger = more spread, in squared units
Std. deviation (σ)	Spread, in original units	$\sigma = \sqrt{Var(X)}$	Typical distance from the mean
Covariance	Do X and Y move together?	$Cov(X,Y) = \frac{1}{n}\sum(x_i - \bar{x})(y_i - \bar{y})$	Positive = move together; negative = move opposite
Correlation (r)	Strength of linear association, scale-free	$r = \frac{Cov(X,Y)}{\sigma_X \sigma_Y}$	Ranges from -1 to 1

Why these matter during exploration: variance tells you if a feature is even useful (a constant feature with ~zero variance carries no information); covariance and correlation tell you which features move together with the target — and which features are redundant with each other.

```
import numpy as np
import pandas as pd

data = pd.DataFrame({"hours": [1, 2, 3, 4, 5], "score": [50, 55, 65, 70, 80]})

mean_hours = data["hours"].mean()
var_hours  = data["hours"].var(ddof=0)      # population variance
std_hours  = data["hours"].std(ddof=0)

covariance  = np.cov(data["hours"], data["score"], ddof=0)[0, 1]
correlation = data["hours"].corr(data["score"])   # Pearson correlation

print(mean_hours, var_hours, std_hours, covariance, correlation)
```

Here, hours and score rise together, so covariance is positive and correlation is close to $+1$ — a strong linear association.

> **Important:** Correlation tells us about **association**, not **causation**. A high correlation between two features doesn't mean one causes the other — both could be driven by a third factor.

We won't go deeper into statistical inference (hypothesis tests, p-values) here — for classification work, this level of descriptive statistics is what you'll use most during exploratory data analysis (EDA).

3. Classification & the Motivation for Logistic Regression

3.1 What Is Classification?

Classification predicts a *category* (a discrete class label), not a continuous number. Regression answers "how much?"; classification answers "which one?"

Problem	Type
Spam / Not Spam	Binary
Fraud / Not Fraud	Binary
Pass / Fail	Binary
Cat / Dog / Horse	Multiclass

- **Binary classification** — exactly two classes (e.g. churn vs. no churn).
- **Multiclass classification** — three or more classes (e.g. species of flower).

3.2 Why Not Just Use Linear Regression?

Linear regression fits a straight line and can output *any* real number. If we tried to use it to predict a class encoded as 0/1, it could easily output values like:

```
-0.4      1.3      2.1
```

None of these are valid probabilities. We need predictions that satisfy:

$$0 \leq p \leq 1$$

so that the output can be interpreted as "the probability of belonging to class 1." This constraint is exactly what **logistic regression** is built to satisfy — by squeezing a linear score through a function that always outputs a value between 0 and 1.

4. Logistic Regression: Mathematics & Intuition

4.1 From Linear Score to Probability

Logistic regression starts exactly like linear regression — with a weighted sum of the features, called the **linear score** or **logit input**:

$$z = b_0 + b_1 x_1 + b_2 x_2 + \cdots + b_p x_p$$

Here z can be any real number, positive or negative, large or small — it's just a weighted combination of inputs plus an intercept b_0.

To turn z into a valid probability, we pass it through the **sigmoid function**:

$$p = \frac{1}{1 + e^{-z}}$$

What the sigmoid does: it "squashes" any real number into the range $(0, 1)$. Very negative z gives $p \approx 0$; very positive z gives $p \approx 1$; $z = 0$ gives exactly $p = 0.5$. This S-shaped curve is why the output can be read directly as a probability.

From probability to class, we apply a threshold (default 0.5):

```
Probability = 0.82
Threshold   = 0.50
Prediction  = Class 1     (since 0.82 >= 0.50)
```

4.2 Odds and Log-Odds (Logit)

Two related quantities help interpret the model:

Odds — the ratio of the probability of the event to the probability of not the event:

$$odds = \frac{p}{1 - p}$$

Log-odds (logit) — the natural log of the odds:

$$\log\left(\frac{p}{1 - p}\right)$$

The key insight of logistic regression: it models the **log-odds as a linear function of the features** — i.e. $\log\left(\frac{p}{1-p}\right) = z = b_0 + b_1 x_1 + \cdots + b_p x_p$. The sigmoid is simply the algebraic inverse of this relationship.

4.3 Interpreting Coefficients

- A **positive coefficient** ($b_j > 0$) increases the log-odds of the positive class as that feature increases.
- A **negative coefficient** ($b_j < 0$) decreases the log-odds of the positive class.
- Each coefficient is interpreted **holding all other features constant**.

Odds ratio. Exponentiating a coefficient gives the odds ratio, which is often easier to communicate:

$$OR = e^{\beta}$$

Example: if the coefficient for "has a support ticket" is $\beta = 0.7$, then $OR = e^{0.7} \approx 2.01$ — customers with a support ticket have roughly **double** the odds of churning, holding other features fixed.

5. How Logistic Regression Learns, and Python Implementation

5.1 The Loss Function: Log Loss

Logistic regression is trained by finding the coefficients that make the model's predicted probabilities match the true labels as closely as possible. The function it minimizes is **binary cross-entropy**, also called **log loss**:

$$L = -\frac{1}{n}\sum_{i=1}^{n}\Big[y_i \log(p_i) + (1-y_i)\log(1-p_i)\Big]$$

Why this loss exists: for each row, only one of the two terms is "active" (since y_i is 0 or 1), and it measures how far the predicted probability was from the true label. **Confident wrong predictions are penalized heavily** — e.g. predicting $p = 0.99$ for an actual class-0 row produces a very large loss, because $-\log(1-0.99) = -\log(0.01)$ is large. Training simply searches for the coefficients that make the total loss as small as possible.

5.2 Full Implementation

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

df = pd.read_csv("dataset.csv")

X = df[["feature1", "feature2"]]
y = df["target"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,       # hold out 20% of the data for testing
    random_state=42,     # makes the split reproducible
    stratify=y           # keeps the same class proportions in train and test
)

model = LogisticRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)
```

Line-by-line, concisely:

- **Imports** bring in `train_test_split` (to create honest train/test sets) and `LogisticRegression` (the model itself).
- **X / y separation** exists because scikit-learn expects features and target to be passed separately to `.fit()`.
- **`train_test_split()`** simulates "unseen data" so we can honestly evaluate the model instead of testing it on the same rows it learned from.
- **`stratify=y`** keeps the class ratio (e.g. 80% no-churn / 20% churn) consistent across train and test — critical when classes are imbalanced.
- **`fit()`** is where learning happens: it finds the coefficients that minimize log loss on the training data.
- **`predict()`** applies the default 0.5 threshold and returns the final predicted **class**.
- **`predict_proba()`** returns the underlying **probabilities** for each class — more informative and adjustable.

Method	Returns
`predict()`	Final class label (e.g. `0` or `1`)
`predict_proba()`	Probability for each class (e.g. `[0.18, 0.82]`)

6. Binary vs. Multiclass Logistic Regression

6.1 Binary Logistic Regression

The simplest and most common case — exactly two outcomes, e.g.:

```
0 → No Churn
1 → Churn
```

The model outputs a single probability, e.g. $P(\text{churn}) = 0.83$, and applies a threshold:

```
p >= 0.5  → class 1
p <  0.5  → class 0
```

Why 0.5 isn't always optimal: the best threshold depends on the *cost* of each mistake. In fraud detection, missing a fraud case (false negative) is usually far more costly than a false alarm, so you might lower the threshold to catch more positives — trading precision for recall (covered in Section 7).

6.2 Multiclass Classification

When there are three or more classes, e.g.:

```
0 → Setosa
1 → Versicolor
2 → Virginica
```

Two common strategies extend logistic regression to multiple classes:

- **One-vs-Rest (OvR)** — trains one binary classifier per class ("this class" vs. "all others") and picks the class with the highest score.
- **Multinomial logistic regression** — generalizes the model directly to multiple classes in one unified formulation, using the **softmax** function instead of the sigmoid.

Softmax converts a vector of raw class scores into probabilities that sum to 1:

$$P(y = k) = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

Each class gets its own linear score z_k; softmax normalizes them into a proper probability distribution over all classes.

```
model = LogisticRegression()          # scikit-learn handles multiclass automatically
model.fit(X_train, y_train)

predictions   = model.predict(X_test)         # single class label per row
probabilities = model.predict_proba(X_test)   # one probability per class, per row
```

6.3 Binary vs. Multiclass at a Glance

Aspect	Binary	Multiclass
Number of classes	2	3+
Activation	Sigmoid	Softmax
`predict()` output	One of two labels	One of k labels
`predict_proba()` shape	`(n_samples, 2)`	`(n_samples, k)`
Typical strategy	Direct sigmoid	One-vs-Rest or multinomial

7. Model Evaluation & the Confusion Matrix

7.1 The Confusion Matrix

Every classification prediction falls into one of four buckets. Using a **fraud detection** example (positive = "fraud"):

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP) — fraud correctly caught	False Negative (FN) — fraud missed
Actual Negative	False Positive (FP) — legit flagged as fraud	True Negative (TN) — legit correctly cleared

7.2 Core Metrics

Metric	Formula	Question it answers
Accuracy	$\frac{TP + TN}{TP + TN + FP + FN}$	How many total predictions were correct?
Precision	$\frac{TP}{TP + FP}$	How trustworthy are positive predictions?
Recall	$\frac{TP}{TP + FN}$	How many actual positives did we find?
Specificity	$\frac{TN}{TN + FP}$	How well do we identify true negatives?
F1 Score	$2 \cdot \frac{Precision \times Recall}{Precision + Recall}$	Balance between precision and recall

Why accuracy can mislead: with imbalanced data (e.g. 99% legitimate transactions, 1% fraud), a model that predicts "not fraud" every single time scores 99% accuracy while catching zero fraud. This is why precision, recall, and F1 matter more when classes are imbalanced.

7.3 ROC-AUC

Two more rates describe how a classifier trades off catching positives against false alarms as the threshold varies:

$$TPR = \frac{TP}{TP + FN} \qquad FPR = \frac{FP}{FP + TN}$$

The **ROC curve** plots TPR against FPR across all possible thresholds; the **AUC** (Area Under the Curve) condenses this into a single number between 0.5 (no better than random) and 1.0 (perfect separation) — it approximates the probability that the model ranks a random positive example above a random negative one.

> For **strongly imbalanced** positive classes, a **Precision-Recall curve** is often more informative than ROC-AUC, since ROC can look overly optimistic when negatives vastly outnumber positives.

```
from sklearn.metrics import (
    confusion_matrix, accuracy_score, precision_score,
    recall_score, f1_score, roc_auc_score
)

cm = confusion_matrix(y_test, y_pred)
print("Accuracy :", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall   :", recall_score(y_test, y_pred))
print("F1       :", f1_score(y_test, y_pred))
print("ROC-AUC  :", roc_auc_score(y_test, y_prob[:, 1]))
```

8. Overfitting, Regularization & the Practical Workflow

8.1 Underfitting vs. Overfitting

Scenario	Training performance	Testing performance	Meaning
Underfitting	Poor	Poor	Model is too simple to capture important patterns
Good fit	Good	Good	Model generalizes well to unseen data
Overfitting	Very good	Poor	Model memorized training data instead of learning general patterns

8.2 Regularization

Regularization adds a penalty for large coefficients to the loss function, discouraging the model from becoming overly complex:

$$\text{L1: } Loss + \lambda \sum |\beta_j| \qquad\qquad \text{L2: } Loss + \lambda \sum \beta_j^2$$

- **L1 (Lasso)** can shrink some coefficients all the way to **zero** — effectively performing feature selection.
- **L2 (Ridge)** shrinks all coefficients smoothly toward zero without eliminating them.
- Both control model complexity and reduce overfitting.

In scikit-learn, regularization strength is controlled by the parameter `C`:

```
LogisticRegression(C=1.0)
```

```
Small C  → Stronger regularization (simpler model)
Large C  → Weaker regularization (closer to unregularized fit)
```

8.3 The Practical Kaggle Workflow

```
Kaggle Dataset
      |
Load with Pandas
      |
head() / info() / describe()
      |
Understand features + target
      |
Check missing values → Clean data
      |
Check class distribution
      |
Select X and y
      |
Train/Test Split (stratified)
      |
Preprocessing (scaling, encoding)
      |
Logistic Regression → fit()
      |
predict() / predict_proba()
      |
Confusion Matrix
      |
Accuracy / Precision / Recall / F1
      |
ROC-AUC (if appropriate)
      |
Check Overfitting (train vs. test scores)
      |
```

```
Tune Regularization (C) / Threshold
      |
Final Model
```

Each step exists to answer one question: *does this model generalize, and is it making the right kind of mistakes for the problem at hand?*

9. Mini Project: Customer Churn Prediction

This walks the full workflow end-to-end on a small, reproducible churn dataset.

```python
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    confusion_matrix, accuracy_score, precision_score,
    recall_score, f1_score, roc_auc_score
)

# 1. Load data
df = pd.read_csv("churn.csv")

# 2-4. First look
df.head()
df.info()
df.describe()

# 5. Handle missing values
df["tenure_months"] = df["tenure_months"].fillna(df["tenure_months"].median())
df = df.dropna(subset=["churn"])

# 6. Feature / target selection
X = df[["tenure_months", "monthly_charges", "support_tickets", "contract_months"]]
y = df["churn"]                      # 0 = stayed, 1 = churned

# 7. Train/test split with stratification
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 8. Basic preprocessing — scale numeric features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)

# 9. Logistic Regression
model = LogisticRegression(C=1.0)
model.fit(X_train_scaled, y_train)

# 10-11. Prediction and probability
y_pred = model.predict(X_test_scaled)
y_prob = model.predict_proba(X_test_scaled)[:, 1]

# 12-17. Evaluation
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Accuracy :", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall   :", recall_score(y_test, y_pred))
print("F1       :", f1_score(y_test, y_pred))
print("ROC-AUC  :", roc_auc_score(y_test, y_prob))

# 19. Check overfitting: compare train vs. test accuracy
```

```
train_acc = model.score(X_train_scaled, y_train)
test_acc  = model.score(X_test_scaled, y_test)
print("Train accuracy:", train_acc, "| Test accuracy:", test_acc)

# 20. Tune C (regularization strength)
for c in [0.01, 0.1, 1, 10]:
    m = LogisticRegression(C=c).fit(X_train_scaled, y_train)
    print(f"C={c}: test accuracy = {m.score(X_test_scaled, y_test):.3f}")

# 21. Predict for one new customer
new_customer = pd.DataFrame([{
    "tenure_months": 3, "monthly_charges": 85,
    "support_tickets": 4, "contract_months": 1
}])
new_scaled = scaler.transform(new_customer)
print("Churn probability:", model.predict_proba(new_scaled)[0, 1])
```

Interpretation. If train accuracy is much higher than test accuracy, the model is overfitting — try a smaller `C` (stronger regularization) or fewer/simpler features. If precision is low, too many loyal customers are being flagged as churn risks; if recall is low, actual churners are slipping through — pick the metric that matches the business cost of each mistake, and adjust the decision threshold on `predict_proba()` accordingly rather than always using 0.5.

10. Cheat Sheets

10.1 Mathematical Formula Sheet

Concept	Formula
Mean	$\bar{x} = \frac{1}{n}\sum x_i$
Variance	$Var(X) = \frac{1}{n}\sum(x_i - \bar{x})^2$
Standard deviation	$\sigma = \sqrt{Var(X)}$
Covariance	$Cov(X,Y) = \frac{1}{n}\sum(x_i - \bar{x})(y_i - \bar{y})$
Correlation	$r = \frac{Cov(X,Y)}{\sigma_X \sigma_Y}$
Linear score	$z = b_0 + b_1 x_1 + \cdots + b_p x_p$
Sigmoid	$p = \frac{1}{1+e^{-z}}$
Odds	$odds = \frac{p}{1-p}$
Logit (log-odds)	$\log\left(\frac{p}{1-p}\right)$
Log loss	$L = -\frac{1}{n}\sum[y_i \log(p_i) + (1-y_i)\log(1-p_i)]$
Accuracy	$\frac{TP+TN}{TP+TN+FP+FN}$
Precision	$\frac{TP}{TP+FP}$
Recall	$\frac{TP}{TP+FN}$
Specificity	$\frac{TN}{TN+FP}$
F1 Score	$2 \cdot \frac{Precision \times Recall}{Precision + Recall}$
TPR	$\frac{TP}{TP+FN}$
FPR	$\frac{FP}{FP+TN}$
Softmax	$P(y=k) = \frac{e^{z_k}}{\sum_j e^{z_j}}$
L1 regularization	$Loss + \lambda\sum\lvert\beta_j\rvert$
L2 regularization	$Loss + \lambda\sum\beta_j^2$

10.2 Python Function Cheat Sheet

Function	Purpose
`pd.read_csv()`	Load dataset
`df.head()`	Preview rows
`df.info()`	Structure, types, null info
`df.describe()`	Statistical summary
`df.drop()`	Remove rows/columns
`df.isnull()`	Detect missing data
`df.fillna()`	Handle missing data
`train_test_split()`	Create train/test sets
`LogisticRegression()`	Classification model
`StandardScaler()`	Feature scaling
`Pipeline()`	Combine preprocessing + model

Function	Purpose
`fit()`	Learn model parameters
`predict()`	Predict classes
`predict_proba()`	Predict probabilities
`confusion_matrix()`	Analyze classification errors
`accuracy_score()`	Accuracy
`precision_score()`	Precision
`recall_score()`	Recall
`f1_score()`	F1
`roc_auc_score()`	ROC-AUC

10.3 Decision-Making Cheat Sheet

Situation	What to consider
Two classes	Binary classification
More than two classes	Multiclass classification
Need probability, not just label	`predict_proba()`
False positives costly	Focus on precision
False negatives costly	Focus on recall
Classes highly imbalanced	Don’t rely only on accuracy
Model overfits	Regularization / simpler model
Need feature selection	Consider L1
Need coefficient shrinkage	Consider L2
Features have very different scales	Consider `StandardScaler`
Preprocessing + model together	Use `Pipeline`
Need threshold adjustment	Use predicted probabilities, not just `predict()`

End of handbook — you now have everything needed to take a new Kaggle classification dataset from raw CSV to an evaluated, tuned logistic regression model.